

Министерство образования Республики Беларусь  
Учреждение образования  
«Брестский государственный технический университет»  
Кафедра ИИТ

Лабораторная работа №1

**По дисциплине:** «Обработка изображений в ИС»

**Тема:** «Обучение классификаторов средствами библиотеки PyTorch»

**Выполнил:**  
Студент 4 курса  
Группы ИИ-26  
Турич Д.А.  
**Проверила:**  
Андренко К.В.

**Цель:** научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

### Общее задание

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

| № варианта | Выборка       | Размер исходного изображения | Оптимизатор |
|------------|---------------|------------------------------|-------------|
| 7          | Fashion-MNIST | 28X28                        | Adam        |

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

```
import matplotlib.pyplot as plt
import numpy as np
import random
import os
from PIL import Image
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
BATCH_SIZE = 64
TEST_BATCH_SIZE = 1000
EPOCHS = 10
LEARNING_RATE = 0.001
RANDOM_SEED = 42
```

```
torch.manual_seed(RANDOM_SEED)
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)
```

```
CLASSES = [
    "Футболка/топ",
    "Брюки",
    "Пуловер",
    "Платье",
    "Пальто",
    "Сандалии",
    "Рубашка",
    "Кроссовки",
    "Сумка",
    "Ботинки"
```

```
]
```

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))
])
```

```
train_dataset = datasets.FashionMNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)
```

```
test_dataset = datasets.FashionMNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)
```

```
train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True
)
```

```
test_loader = DataLoader(
    test_dataset,
    batch_size=TEST_BATCH_SIZE,
    shuffle=False
)
```

```
class FashionCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(
            in_channels=1,
            out_channels=16,
            kernel_size=3,
            padding=1
        )

        self.conv2 = nn.Conv2d(
            in_channels=16,
            out_channels=32,
            kernel_size=3,
            padding=1
        )

        self.pool = nn.MaxPool2d(
            kernel_size=2,
            stride=2
        )

        self.fc1 = nn.Linear(
            32 * 7 * 7,
```

```

        128
    )

    self.fc2 = nn.Linear(
        128,
        num_classes
    )

    self.dropout = nn.Dropout(0.25)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))

        x = x.view(x.size(0), -1)

        x = F.relu(self.fc1(x))
        x = self.dropout(x)

        x = self.fc2(x)

        return x

model = FashionCNN().to(DEVICE)

print(model)
print()
print("Устройство:", DEVICE)
print("Количество обучающих изображений:", len(train_dataset))
print("Количество тестовых изображений:", len(test_dataset))
print()

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=LEARNING_RATE
)

def train_one_epoch(epoch):
    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for batch_idx, (data, target) in enumerate(train_loader):

        data = data.to(DEVICE)
        target = target.to(DEVICE)

        optimizer.zero_grad()

        output = model(data)

        loss = criterion(output, target)

        loss.backward()

```

```

optimizer.step()

running_loss += loss.item()

predicted = output.argmax(dim=1)

total += target.size(0)
correct += predicted.eq(target).sum().item()

if batch_idx % 200 == 0:
    print(
        f"Эпоха {epoch} "
        f"[{batch_idx * len(data)} / {len(train_loader.dataset)}] "
        f"Loss: {loss.item():.4f}"
    )

average_loss = running_loss / len(train_loader)
accuracy = 100.0 * correct / total

return average_loss, accuracy

def evaluate(loader):
    model.eval()

    total_loss = 0.0
    correct = 0

    with torch.no_grad():

        for data, target in loader:

            data = data.to(DEVICE)
            target = target.to(DEVICE)

            output = model(data)

            total_loss += (
                criterion(output, target).item()
                * data.size(0)
            )

            predicted = output.argmax(dim=1)

            correct += predicted.eq(target).sum().item()

    average_loss = total_loss / len(loader.dataset)

    accuracy = 100.0 * correct / len(loader.dataset)

    return average_loss, accuracy

train_losses = []
test_losses = []

train_accuracies = []
test_accuracies = []

```

```

print("Начало обучения")
print("=" * 60)

for epoch in range(1, EPOCHS + 1):

    train_loss, train_acc = train_one_epoch(epoch)

    test_loss, test_acc = evaluate(test_loader)

    train_losses.append(train_loss)
    test_losses.append(test_loss)

    train_accuracies.append(train_acc)
    test_accuracies.append(test_acc)

    print(
        f"\nЭпоха {epoch}: "
        f"train_loss={train_loss:.4f}, "
        f"train_acc={train_acc:.2f}%, "
        f"test_loss={test_loss:.4f}, "
        f"test_acc={test_acc:.2f}%"
    )

    print("=" * 60)

print()
print(
    f"Итоговая точность на тестовой выборке: "
    f"{test_accuracies[-1]:.2f}%"
)

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPOCHS + 1),
    train_losses,
    label="Train loss",
    marker="o"
)

plt.plot(
    range(1, EPOCHS + 1),
    test_losses,
    label="Test loss",
    marker="o"
)

plt.xlabel("Эпоха")
plt.ylabel("Loss")
plt.title("Изменение ошибки при обучении CNN")
plt.legend()
plt.grid(True)

plt.savefig(
    "loss_curve.png",
    dpi=150,
    bbox_inches="tight"
)

```

```

plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPOCHS + 1),
    train_accuracies,
    label="Train accuracy",
    marker="o"
)

plt.plot(
    range(1, EPOCHS + 1),
    test_accuracies,
    label="Test accuracy",
    marker="o"
)

plt.xlabel("Эпоха")
plt.ylabel("Точность, %")
plt.title("Изменение точности при обучении CNN")
plt.legend()
plt.grid(True)

plt.savefig(
    "accuracy_curve.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

```

```

def visualize_prediction(model, dataset, index=None):

    model.eval()

    if index is None:
        index = random.randint(
            0,
            len(dataset) - 1
        )

    image, true_label = dataset[index]

    input_tensor = image.unsqueeze(0).to(DEVICE)

    with torch.no_grad():

        output = model(input_tensor)

        probabilities = F.softmax(
            output,
            dim=1
        ).cpu().numpy().flatten()

        predicted_label = int(
            np.argmax(probabilities)
        )

```

```

img_np = (
    image.squeeze().numpy()
    * 0.3530
    + 0.2860
)

fig, axes = plt.subplots(
    1,
    2,
    figsize=(11, 4)
)

axes[0].imshow(
    img_np,
    cmap="gray"
)

axes[0].set_title(
    f"Истинный класс:\n"
    f"{CLASSES[true_label]}\n\n"
    f"Предсказание:\n"
    f"{CLASSES[predicted_label]}"
)

axes[0].axis("off")

axes[1].bar(
    range(10),
    probabilities
)

axes[1].set_xticks(range(10))

axes[1].set_xticklabels(
    range(10)
)

axes[1].set_xlabel("Номер класса")
axes[1].set_ylabel("Вероятность")

axes[1].set_title(
    "Распределение вероятностей"
)

plt.tight_layout()

plt.savefig(
    "prediction_example.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

print()
print("Результат классификации")
print("-" * 40)
print("Индекс изображения:", index)
print(
    "Истинный класс:",

```

```

    CLASSES[true_label]
)
print(
    "Предсказанный класс:",
    CLASSES[predicted_label]
)
print(
    "Вероятность предсказанного класса:",
    f"{probabilities[predicted_label] * 100:.2f}%"
)

print()
print("Вероятности по классам:")

for i in range(10):
    print(
        f"{i} - {CLASSES[i]:15s}: "
        f"{probabilities[i] * 100:.2f}%"
    )

```

```

def load_custom_image(path):

    image = Image.open(path).convert("L")

    image = image.resize(
        (28, 28)
    )

    img_array = (
        np.array(image)
        .astype(np.float32)
        / 255.0
    )

    img_array = (
        (img_array - 0.2860)
        / 0.3530
    )

    tensor = torch.tensor(
        img_array,
        dtype=torch.float32
    )

    tensor = tensor.unsqueeze(0).unsqueeze(0)

    return tensor, img_array

```

```

def predict_custom_image(model, path):

    model.eval()

    tensor, img_array = load_custom_image(path)

    tensor = tensor.to(DEVICE)

    with torch.no_grad():

        output = model(tensor)

```

```

probabilities = F.softmax(
    output,
    dim=1
).cpu().numpy().flatten()

predicted_label = int(
    np.argmax(probabilities)
)

fig, axes = plt.subplots(
    1,
    2,
    figsize=(11, 4)
)

axes[0].imshow(
    img_array,
    cmap="gray"
)

axes[0].set_title(
    f"Произвольное изображение\n"
    f"Предсказание: "
    f"{CLASSES[predicted_label]}"
)

axes[0].axis("off")

axes[1].bar(
    range(10),
    probabilities
)

axes[1].set_xticks(range(10))

axes[1].set_xlabel("Номер класса")
axes[1].set_ylabel("Вероятность")

axes[1].set_title(
    "Распределение вероятностей"
)

plt.tight_layout()

plt.savefig(
    "custom_prediction.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

print()
print("Распознавание произвольного изображения")
print("-" * 40)
print("Файл:", path)
print(
    "Предсказанный класс:",
    CLASSES[predicted_label]
)

```

```

print(
    "Вероятность:",
    f"{probabilities[predicted_label] * 100:.2f}%"
)

```

```

visualize_prediction(
    model,
    test_dataset
)

```

```

custom_image_path = "my_clothing.png"

```

```

if os.path.exists(custom_image_path):

```

```

    predict_custom_image(
        model,
        custom_image_path
    )

```

```

else:

```

```

    print()
    print(
        f"Файл {custom_image_path} не найден."
    )

```

```

    print(
        "Для проверки своей картинки "
        "положите её рядом с программой "
        "и назовите my_clothing.png."
    )

```

```

Эпоха 9 [0/60000] Loss: 0.1147
Эпоха 9 [12800/60000] Loss: 0.0682
Эпоха 9 [25600/60000] Loss: 0.1293
Эпоха 9 [38400/60000] Loss: 0.0930
Эпоха 9 [51200/60000] Loss: 0.2734

Эпоха 9: train_loss=0.1690, train_acc=93.67%, test_loss=0.2401, test_acc=91.43%
=====
Эпоха 10 [0/60000] Loss: 0.2117
Эпоха 10 [12800/60000] Loss: 0.1814
Эпоха 10 [25600/60000] Loss: 0.0725
Эпоха 10 [38400/60000] Loss: 0.0735
Эпоха 10 [51200/60000] Loss: 0.1829

Эпоха 10: train_loss=0.1567, train_acc=94.11%, test_loss=0.2484, test_acc=91.78%
=====

Итоговая точность на тестовой выборке: 91.78%
)

```

# Результат классификации

Индекс изображения: 1824

Истинный класс: Сандалии

Предсказанный класс: Сандалии

Вероятность предсказанного класса: 100.00%

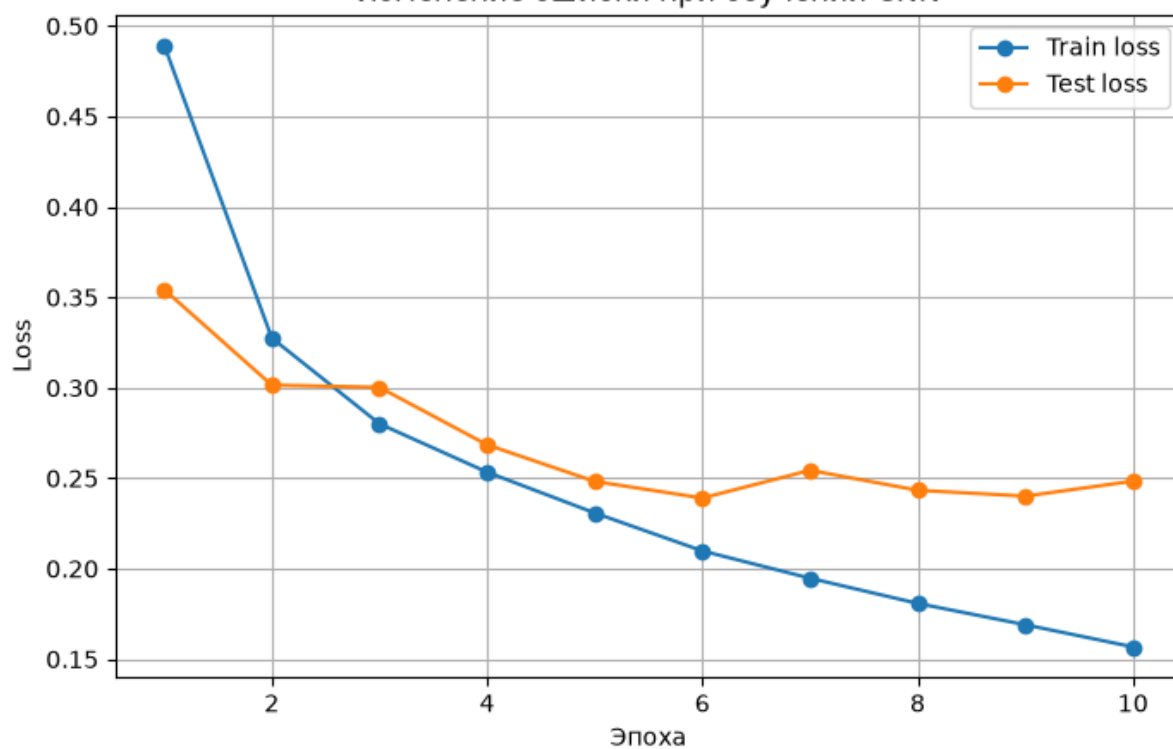
## Вероятности по классам:

|                  |           |
|------------------|-----------|
| 0 - Футболка/топ | : 0.00%   |
| 1 - Брюки        | : 0.00%   |
| 2 - Пуловер      | : 0.00%   |
| 3 - Платье       | : 0.00%   |
| 4 - Пальто       | : 0.00%   |
| 5 - Сандалии     | : 100.00% |
| 6 - Рубашка      | : 0.00%   |
| 7 - Кроссовки    | : 0.00%   |
| 8 - Сумка        | : 0.00%   |
| 9 - Ботинки      | : 0.00%   |

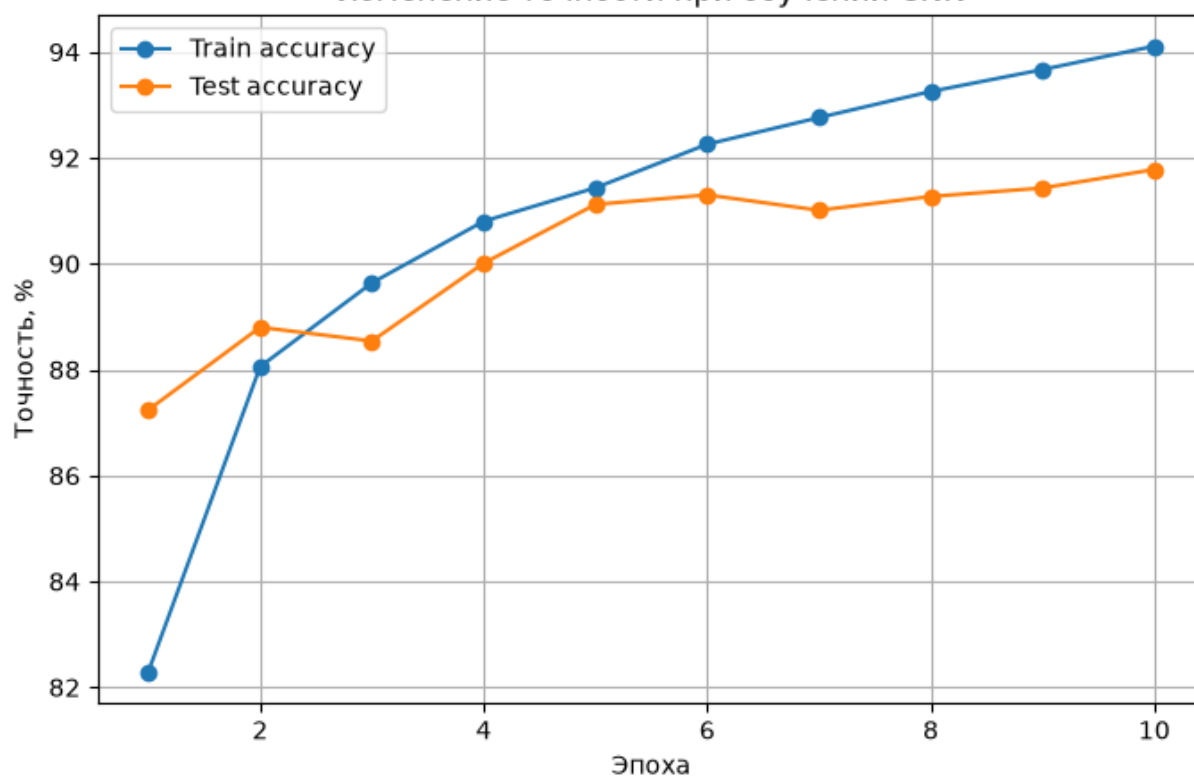
Файл my\_clothing.png не найден.

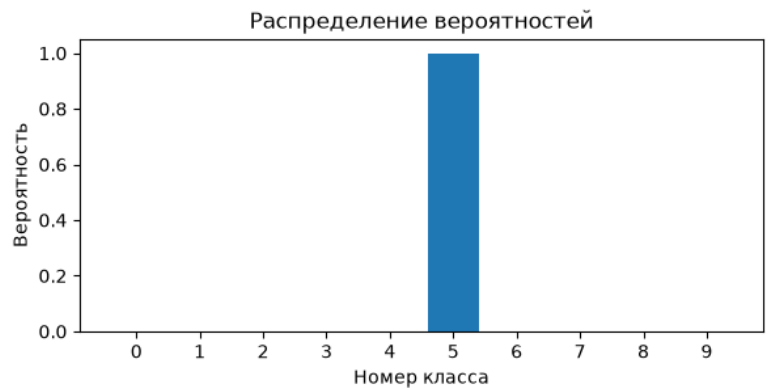
Для проверки своей картинке положите её рядом с программой и назовите my\_clothing.png.

Изменение ошибки при обучении CNN



Изменение точности при обучении CNN





Сеть успешно обучилась: итоговая точность на тестовой выборке составила 91.78%. Данные о динамике ошибки (loss) и количестве эпох на скрине отсутствуют, однако достигнутый уровень точности свидетельствует о хорошем качестве модели.

На тестовом изображении сеть дала верное и уверенное предсказание с вероятностью 100.00%. При этом проверить собственное нарисованное изображение не удалось: файл `my_clothing.png` не был найден, поэтому программа не смогла выполнить предсказание для него и вывела соответствующее предупреждение.

**Вывод:** Изучил принципы бинарной классификации и реализовал однослойную нейронную сеть (персептрон) для решения задачи классификации с использованием **пороговой** функции активации, а также исследовал процесс обучения модели с применением среднеквадратичной ошибки (MSE).